CMSC132: Introduction to Data Structures

Chapter 9

By Naren Ayinala

May 2026

Contents

1 Why Do We Need Data Structures? **2**

2 Abstract Data Types vs Implementations **2**

3 Linear vs Nonlinear Structures **2**

4 Arrays and Dynamic Arrays **3**

5 Stacks: Last In, First Out **3**

6 Queues: First In, First Out **4**

7 Sets, Maps, Trees, and Heaps at a Glance **4**

8 How Do We Compare Structures? **4**

9 Choosing a Structure: Worked Scenarios **5**

10 Iterators and Safe Traversal **5**

11 Practice Questions and Answers **5**

1 Why Do We Need Data Structures?

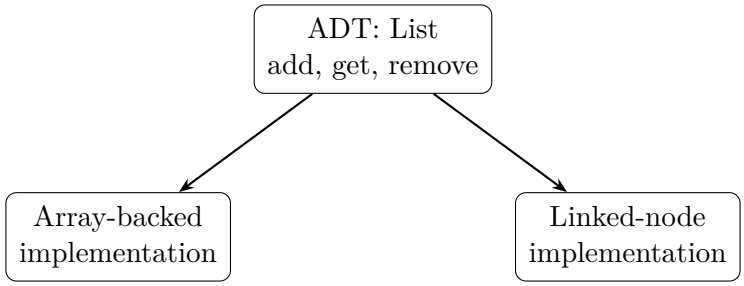
Imagine you are writing a program that stores the grades of ten students. An array might work perfectly. Now imagine the program must store thousands of students, search for one by ID, insert a new student, and remove someone who dropped the course. The choice of how we organize the data begins to matter a lot.

A **data structure** is a way to organize and store data so that we can perform operations on it efficiently. The important word is *operations*. Two structures can store the same values yet make different operations easier or harder.

Example: Suppose we want to look up a student's grade by student ID. Searching a plain unsorted array might require checking every entry. A hash map often gives expected constant-time lookup. However, if we want to print all students in sorted ID order, a tree-based map might be a better choice.

2 Abstract Data Types vs Implementations

An **abstract data type (ADT)** describes *what* operations a structure supports, not the precise code used to implement it. A `List` ADT supports adding, retrieving, and removing elements by position. A `Stack` ADT supports push, pop, and peek. The implementation might use arrays or linked nodes.



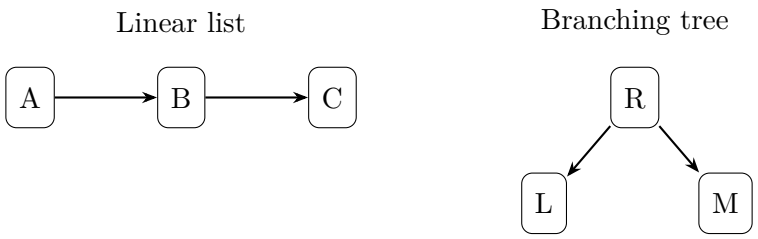
Important distinction: `List` is an interface in Java. `ArrayList` and `LinkedList` are concrete implementations. This is an example of programming to an interface instead of tying code to one implementation.

```
import java.util.*;
List<String> names = new ArrayList<>();
names.add("Ada");
names.add("Grace");
System.out.println(names.get(0)); // Ada

// We can swap implementations without changing the List API:
List<String> other = new LinkedList<>();
```

3 Linear vs Nonlinear Structures

A **linear** structure organizes items in a sequence: arrays, lists, stacks, and queues. A **nonlinear** structure models branching or relationships: trees, heaps, and graphs.



4 Arrays and Dynamic Arrays

An array has a fixed length after creation. Java arrays provide constant-time indexing because the location of an element can be computed from its index. A dynamic array, such as `ArrayList`, grows when needed by allocating a larger backing array and copying references.

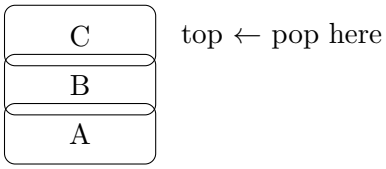
Why does adding at the end usually feel fast? Most calls to `add` put the new element into a free slot. Occasionally the backing array must grow and copy all existing elements. This makes a single resize expensive, but *amortized* append time remains $O(1)$.

```
int[] fixed = new int[3];
fixed[0] = 10;

ArrayList<Integer> dynamic = new ArrayList<>();
dynamic.add(10);
dynamic.add(20);
dynamic.add(1, 15); // shifts 20 right
System.out.println(dynamic); // [10, 15, 20]
```

5 Stacks: Last In, First Out

A **stack** is **LIFO**: the last element pushed is the first element popped. Think of a stack of trays: you normally remove the top tray first. Useful applications include undo operations, call stacks, and depth-first search.




```
Deque<Integer> stack = new ArrayDeque<>();
stack.push(10);
stack.push(20);
System.out.println(stack.peek()); // 20
System.out.println(stack.pop()); // 20
System.out.println(stack.pop()); // 10
```

Common trap: `peek()` reads the top without removing it, while `pop()` removes and returns it. In modern Java, `ArrayDeque` is generally preferred to the legacy `Stack` class.

6 Queues: First In, First Out

A **queue** is **FIFO**: the first item inserted is the first item removed. Think of people waiting in a line. Queues are used in breadth-first search, task scheduling, and event processing.

```
Queue<String> q = new ArrayDeque<>();
q.offer("A");
q.offer("B");
System.out.println(q.peek()); // A
System.out.println(q.poll()); // A
System.out.println(q.poll()); // B
System.out.println(q.poll()); // null (empty)
```

Note: `offer` adds, `poll` removes, and `peek` examines. On an empty queue, `poll` and `peek` return `null`, whereas `remove` and `element` throw exceptions.

7 Sets, Maps, Trees, and Heaps at a Glance

A **set** stores unique elements. A **map** stores key–value associations and unique keys. A **binary search tree** organizes comparable keys so smaller keys appear to the left and larger keys to the right (under a no-duplicates convention). A **heap** ensures that each parent's priority is ordered relative to its children; unlike a BST, a heap does not maintain a fully sorted left/right search arrangement.

```
Set<Integer> ids = new HashSet<>();
ids.add(42);
ids.add(42); // duplicate ignored

Map<String, Integer> scores = new HashMap<>();
scores.put("Ada", 97);
scores.put("Grace", 94);
System.out.println(scores.get("Ada")); // 97

PriorityQueue<Integer> minHeap = new PriorityQueue<>();
minHeap.offer(9);
minHeap.offer(2);
System.out.println(minHeap.poll()); // 2
```

8 How Do We Compare Structures?

We often compare **time complexity** and **space complexity**. Big-O describes an asymptotic upper bound; it is not a precise count of milliseconds. In the table below, n is the number of elements. The expected complexity for hashing assumes a reasonably distributed hash function and an appropriate implementation.

Structure	Typical operation	Complexity
Array	Index lookup	$O(1)$
Unsorted array/list	Find by value	$O(n)$
ArrayList	Append	Amortized $O(1)$
ArrayList	Insert near front	$O(n)$
Linked list	Insert after known node	$O(1)$
Stack / queue	Push/pop or offer/poll	$O(1)$
HashMap	Lookup	Expected $O(1)$
Balanced BST	Lookup	$O(\log n)$
Binary heap	Insert / remove root	$O(\log n)$

Caution: Finding the node in a linked list may still take $O(n)$; inserting after a *known* node is the constant-time part. A hash map does not guarantee $O(1)$ for every input.

9 Choosing a Structure: Worked Scenarios

1. **Undo button:** A stack is natural because the most recent action is undone first.
2. **Customers in arrival order:** A queue preserves FIFO processing.
3. **Unique course codes:** A set avoids duplicates.
4. **Student ID → grade:** A map models the key–value relationship.
5. **Always process the smallest deadline first:** A priority queue backed by a heap.
6. **Find all IDs in sorted order:** A tree-based ordered set or map can help.

10 Iterators and Safe Traversal

An iterator lets us visit a collection without exposing its internal representation. This matters because an `ArrayList` and `LinkedList` store elements differently but can both be traversed with an iterator.

```
List<String> names = new ArrayList<>(List.of("A", "B", "C"));
Iterator<String> it = names.iterator();
while (it.hasNext()) {
    String s = it.next();
    if (s.equals("B")) it.remove(); // safe iterator removal
}
System.out.println(names); // [A, C]
```

Do not generally modify a collection structurally with `list.remove(...)` while traversing it with a fail-fast iterator; use the iterator's own `remove()` where supported.

11 Practice Questions and Answers

Q1. What is the difference between an ADT and an implementation?
Answer: The ADT specifies behavior; an implementation specifies how that behavior is achieved.

Q2. A stack receives 4, 7, 9 in that order. What do three pops return?
Answer: 9, 7, 4.

Q3. Why is insertion at the front of an `ArrayList` typically $O(n)$?
Answer: Existing elements must be shifted right to make room.

Q4. Which structure should hold unique usernames?
Answer: A set, often a `HashSet` if ordering is unnecessary.

Q5. Does a heap let us binary-search arbitrary values?
Answer: No. Its ordering guarantee is between each parent and child, not a global BST order.